Conditional infit versus outfit under family-wise error control

Dichotomous and polytomous items, B = 1000, Westfall-Young

Magnus Johansson

2026-08-03

Table of contents

Scope

Every study in this paper decides on conditional **infit**. The parametric bootstrap in `RMitemInfitCutoff()` simulates outfit at the same time and stores it, and `RMitemInfit()` gained a `statistic` argument (easyRasch2 > 1.1.0) that reports and tests it through the same Westfall-Young step-down. So the comparison costs one extra call per bootstrap and no extra simulation.

The question is narrow. Holding the decision rule fixed at `correction = "fwer"`, `alpha = .05` and `B = 1000`, does outfit control the family-wise error rate as infit does, and does it detect anything infit misses?

Outfit is the unweighted mean of squared standardised residuals, so it is dominated by responses far from the item location. If it earns a place beside infit anywhere, it is on an **off-target** item. That contrast is the spine of the design: within each arm one misfitting item sits on the person mean and one sits well above it, run as separate conditions rather than nested, so detection is attributable to targeting rather than to the number of misfitting items.

Both statistics come from the same bootstrap

Within a replication the data, the item parameters, the bootstrap draws and the step-down are shared. Only the column read from `$results` differs (`InfitMSQ` or `OutfitMSQ`). Every infit-versus-outfit comparison below is therefore paired at the replication level, which is what makes the discordance counts in the last section interpretable.

Misfit calibrated on the infit scale, not on rho

Following the Method section, `rho` is not portable across item formats, so the two arms are anchored to a comparable **infit** level rather than to a common `rho`. The target is the range of the items that the real phq9 responses flag, roughly 1.23 to 1.32.

A misfitting item far from the person mean injects less infit than an on-target one at the same `rho`, so the anchoring has to hold at both targeting levels. `calibration_rho_targeting.R` scans it, 80 replications at $n = 600$:

Arm	rho	infit on-target	infit off-target	outfit on-target	outfit off-target
`dich20`	0.50	1.376	1.303	1.743	2.028

Arm	rho	infit on-target	infit off-target	outfit on-target	outfit off-target
`dich20`	**0.60**	**1.304**	**1.243**	1.604	1.735
`dich20`	0.65	1.265	1.216	1.499	1.615
`dich20`	0.75	1.202	1.158	1.362	1.446
`phq9`	**0.85**	**1.271**	**1.264**	1.314	1.388

`rho` = 0.60 and 0.85 are the selected values. They put mean injected infit at 1.274 and 1.268, matched across formats and inside the real flagged range.

Two things in that table are worth carrying into the results. Injected infit falls as the misfitting item moves off target while injected **outfit rises**, which is the asymmetry the study exists to price. And the same infit injection produces a far larger outfit excess in the dichotomous arm (1.60 to 1.74) than in phq9 (1.31 to 1.39). Neither is a prediction about detection, because outfit's null variance is larger too, by roughly a factor of three in the dichotomous arm. Whether the larger signal survives the larger noise is the question.

The study's own first results section reports the achieved values, so the anchoring is verifiable from the run rather than only from the scan. If either arm drifts outside the 1.2 to 1.35 band, add a `rho` level and re-render: `rho` is a per-arm constant here, so widening it means adding it to the grid keys first.

Two arms

	`dich20`	`phq9`
Items	20 dichotomous	9 polytomous, CML from real responses
Locations	`runif(20, -2, 2)`, seed 20250727 (as in the dichotomous studies)	real, range 2.46 logits
Latent SD	1.5	1.39 (estimated)
On-target misfit	V8, location 0.04	q5, location −0.15
Off-target misfit	V9, location 1.82	q8, location 1.40
Generator	`eRm::sim.xdim()`	`sim_partial_score()`

Sample sizes are 250 and 600 for both arms. 600 is also the size of the real phq9 sample the polytomous arm is derived from.

Cost

Three conditions per arm (null, on-target, off-target), two sample sizes, two arms, 250 replications. 3000 cells, each one bootstrap of 1000 iterations plus three `RMitemInfit()` calls. Roughly five hours on ten cores. Cells are ordered so the null conditions and the cheaper sample size finish first, since the null carries the error-rate claim.

```
suppressMessages({
  source("../R/helpers.R")   # package internals and shared helper functions
  library(easyRasch2)
  library(eRm)
  library(MASS)
  library(parallel)
  library(dplyr)
  library(tidyr)
  library(ggplot2)
})

NS      <- c(250, 600)
B       <- 1000          # the decision rule under study
REPS    <- 250
```

```
ALPHA   <- 0.05
NCORES  <- 10
CHUNK   <- 100
RES     <- "infit_outfit.rds"

# This study needs the `statistic` argument, added to RMitemInfit() after
# easyRasch2 1.1.0. Checked here rather than five hours into the run.
if (!"statistic" %in% names(formals(easyRasch2::RMitemInfit))) {
  stop("Installed easyRasch2 (", packageVersion("easyRasch2"), ") has no ",
       "`statistic` argument in RMitemInfit(). Install the development ",
       "version before running this study.", call. = FALSE)
}
```

```
# run_resumable() is defined in ../R/helpers.R
```

Arm definitions

```
# ---- dich20: the item locations used by the dichotomous studies -------------
set.seed(20250727)
D_LOCS <- runif(20, -2, 2)
D_ON   <- which.min(abs(D_LOCS))          # V8,  0.04 logits
D_OFF  <- which.max(D_LOCS)               # V9,  1.82 logits
D_SD   <- 1.5

# ---- phq9: CML estimates from the packaged real data ------------------------
ph <- load_phq9_arm()

ARMS <- list(
  dich20 = list(locs = D_LOCS, sd = D_SD, nm = paste0("V", 1:20),
                on = D_ON, off = D_OFF, rho = 0.60, thr = NULL),
  phq9   = list(locs = ph$locs, sd = ph$sd, nm = paste0("q", 1:9),
                on = 5L, off = 8L, rho = 0.85, thr = ph$thr)
)

bind_rows(lapply(names(ARMS), function(a) {
  x <- ARMS[[a]]
  data.frame(arm = a, items = length(x$locs), rho = x$rho,
             latent_sd = round(x$sd, 2),
             on_target = paste0(x$nm[x$on], " (", round(x$locs[x$on], 2), ")"),
             off_target = paste0(x$nm[x$off], " (", round(x$locs[x$off], 2), ")"))
}))
```

arm	items	rho	latent_sd	on_target	off_target
dich20	20	0.60	1.50	V8 (0.04)	V9 (1.82)
phq9	9	0.85	1.39	q5 (-0.15)	q8 (1.4)

```
# One dataset. `target` is "none" (single dimension), "on" or "off": the named
# item is generated from a second dimension correlating `rho` with the first.
sim_arm <- function(arm, n, target, seed) {
  a <- ARMS[[arm]]
  ni <- length(a$locs)
  mis <- switch(target, none = integer(0), on = a$on, off = a$off)
  set.seed(seed)
```

```
  S <- a$sd^2 * matrix(c(1, a$rho, a$rho, 1), 2, 2)
  th <- MASS::mvrnorm(n, mu = c(0, 0), Sigma = S)

  if (is.null(a$thr)) {
    # dichotomous, via the same generator as the dichotomous studies
    W <- cbind(rep(1, ni), rep(0, ni))
    if (length(mis)) W[mis, ] <- c(0, 1)
    d <- as.data.frame(eRm::sim.xdim(persons = th, items = a$locs,
                                     Sigma = S, weightmat = W))
  } else {
    d <- sim_partial_score(a$thr, th[, 1])
    if (length(mis)) d[, mis] <- sim_partial_score(a$thr[mis], th[, 2])[, 1]
    d <- as.data.frame(d)
  }
  names(d) <- a$nm
  list(data = d, mis = if (length(mis)) mis else 0L)
}
```

Run

One bootstrap per replication, read twice. `RMitemInfit()` is called once without a cutoff for the observed statistics and once per statistic with `p_value = TRUE`, all against the same `RMitemInfitCutoff()` object.

```
one_rep <- function(g) {
  out <- try({
    arm <- as.character(g$arm); target <- as.character(g$target)
    n <- g$n; rep <- g$rep
    ai <- match(arm, names(ARMS)); ti <- match(target, c("none", "on", "off"))
    seed_data <- 4e8 + 1e7 * ai + 1e6 * ti + 1e3 * n + rep

    sd_out <- sim_arm(arm, n, target, seed_data)
    d <- sd_out$data
    items <- names(d)

    co <- RMitemInfitCutoff(d, iterations = B, parallel = FALSE,
                            seed = seed_data + 5e8, verbose = FALSE)

    fi <- suppressWarnings(
      RMitemInfit(d, cutoff = co, p_value = TRUE, correction = "fwer",
                  alpha = ALPHA, statistic = "infit", output = "dataframe"))
    fo <- suppressWarnings(
      RMitemInfit(d, cutoff = co, p_value = TRUE, correction = "fwer",
                  alpha = ALPHA, statistic = "outfit", output = "dataframe"))

    i1 <- match(items, fi$Item); i2 <- match(items, fo$Item)
    data.frame(
      item = seq_along(items),
      mis = sd_out$mis,
      infit_msq  = fi$Infit_MSQ[i1],
      outfit_msq = fo$Outfit_MSQ[i2],
      p_infit    = fi$p_infit[i1],
      padj_infit = fi$padj_infit[i1],
      p_outfit   = fo$p_outfit[i2],
      padj_outfit = fo$padj_outfit[i2],
      iters = co$actual_iterations
    )
  }, silent = TRUE)
```

```
  if (inherits(out, "try-error")) NULL else out
}

# Null conditions first (they carry the error-rate claim), then on-target and
# off-target, cheaper n first within each block.
grid <- do.call(rbind, lapply(c("none", "on", "off"), function(tg) {
  expand.grid(rep = 1:REPS, n = NS, target = tg, arm = names(ARMS),
              stringsAsFactors = FALSE)[, c("arm", "target", "n", "rep")]
}))
grid <- grid[order(match(grid$target, c("none", "on", "off")), grid$n), ]
rownames(grid) <- NULL
cat("cells to run:", nrow(grid), "\n")
```

```
cells to run: 3000
```

```
s <- run_resumable(RES, grid, c("arm", "target", "n", "rep"), one_rep,
                   extra = list(arms = lapply(ARMS, function(a)
                     a[c("locs", "sd", "nm", "on", "off", "rho")]),
                     B = B, alpha = ALPHA))
```

```
running 3000 new cells (0 cached)
```

```
  checkpoint 100 / 3000  (2.7 min)
```

```
  checkpoint 200 / 3000  (5.4 min)
```

```
  checkpoint 300 / 3000  (9 min)
```

```
  checkpoint 400 / 3000  (13.3 min)
```

```
  checkpoint 500 / 3000  (17.6 min)
```

```
  checkpoint 600 / 3000  (21.6 min)
```

```
  checkpoint 700 / 3000  (25.6 min)
```

```
  checkpoint 800 / 3000  (31.3 min)
```

```
  checkpoint 900 / 3000  (38.9 min)
```

```
  checkpoint 1000 / 3000  (46.7 min)
```

```
  checkpoint 1100 / 3000  (49.4 min)
```

```
  checkpoint 1200 / 3000  (52 min)
```

```
  checkpoint 1300 / 3000  (55.5 min)
```

```
  checkpoint 1400 / 3000  (59.8 min)
```

```
  checkpoint 1500 / 3000  (64.1 min)
```

```
  checkpoint 1600 / 3000  (68.1 min)
```

```
  checkpoint 1700 / 3000  (72.1 min)
```

```
  checkpoint 1800 / 3000  (77.9 min)
```

```
  checkpoint 1900 / 3000  (85.5 min)
```

```
  checkpoint 2000 / 3000  (93.1 min)
```

```
  checkpoint 2100 / 3000  (95.8 min)
```

```
  checkpoint 2200 / 3000  (98.4 min)
```

```
  checkpoint 2300 / 3000  (101.9 min)
```

```
  checkpoint 2400 / 3000  (106.2 min)
```

```
  checkpoint 2500 / 3000  (110.4 min)
```

```
  checkpoint 2600 / 3000  (114.4 min)
```

```
  checkpoint 2700 / 3000  (118.4 min)
```

```
  checkpoint 2800 / 3000  (124.2 min)
```

```
  checkpoint 2900 / 3000  (131.8 min)
```

```
  checkpoint 3000 / 3000  (139.4 min)
```

```
R <- do.call(rbind, lapply(seq_len(nrow(s$grid)), function(i) {
  r <- s$res[[i]]
  if (is.null(r)) return(NULL)
```

```
  g <- s$grid[i, c("arm", "target", "n", "rep"), drop = FALSE]
  rownames(g) <- NULL   # otherwise cbind() warns once per cell
  cbind(g, r)
})) |>
  mutate(is_misfit = mis > 0 & item == mis,
         loc = mapply(function(a, i) round(s$arms[[a]]$locs[i], 2), arm, item),
         f_in  = as.integer(padj_infit  < ALPHA),
         f_out = as.integer(padj_outfit < ALPHA),
         f_any = as.integer(f_in == 1 | f_out == 1))

cat(nrow(s$grid), "cells complete |", round(s$elapsed_min, 1), "min\n")
```

```
3000 cells complete | 139.4 min
```

```
cat("replications with a failed cell:", sum(vapply(s$res, is.null, logical(1))), "\n")
```

```
replications with a failed cell: 0
```

Achieved misfit, on both scales

Read this first. The two arms are meant to inject comparable misfit on the infit scale, at roughly the level the real phq9 items reach. The outfit column shows what the same injection does to the unweighted statistic, which is the mechanism the rest of the study prices.

```
R |>
  filter(is_misfit) |>
  group_by(arm, target, loc, n) |>
  summarise(infit = round(mean(infit_msq), 3),
            outfit = round(mean(outfit_msq), 3),
            sd_infit = round(sd(infit_msq), 3),
            sd_outfit = round(sd(outfit_msq), 3), .groups = "drop") |>
  arrange(arm, target, n)
```

arm	target	loc	n	infit	outfit	sd_infit	sd_outfit
dich20	off	1.82	250	1.253	1.836	0.085	0.550
dich20	off	1.82	600	1.242	1.784	0.059	0.350
dich20	on	0.04	250	1.307	1.611	0.088	0.222
dich20	on	0.04	600	1.300	1.577	0.057	0.150
phq9	off	1.40	250	1.261	1.385	0.124	0.266
phq9	off	1.40	600	1.268	1.403	0.072	0.142
phq9	on	−0.15	250	1.274	1.323	0.091	0.121
phq9	on	−0.15	600	1.276	1.322	0.062	0.086

Fitting items under the null, for reference:

```
R |>
  filter(target == "none") |>
  group_by(arm, n) |>
  summarise(infit = round(mean(infit_msq), 3),
```

```
            outfit = round(mean(outfit_msq), 3),
            sd_infit = round(sd(infit_msq), 3),
            sd_outfit = round(sd(outfit_msq), 3), .groups = "drop")
```

arm	n	infit	outfit	sd_infit	sd_outfit
dich20	250	1	0.998	0.074	0.213
dich20	600	1	1.002	0.048	0.137
phq9	250	1	0.997	0.082	0.117
phq9	600	1	1.001	0.053	0.078

Family-wise error under a true model

The primary calibration result. A replication contributes one indicator per rule: at least one item flagged when no item misfits. `union` is the rule an analyst applies by reading both columns of a single table and flagging either, which tests $2k$ hypotheses while each correction covers k.

```
R |>
  filter(target == "none") |>
  group_by(arm, n, rep) |>
  summarise(infit = as.integer(sum(f_in) > 0),
            outfit = as.integer(sum(f_out) > 0),
            union = as.integer(sum(f_any) > 0), .groups = "drop") |>
  group_by(arm, n) |>
  summarise(across(c(infit, outfit, union), \(x) round(100 * mean(x), 1)),
            reps = n(), .groups = "drop") |>
  mutate(mcse = round(100 * sqrt(0.05 * 0.95 / reps), 2))
```

arm	n	infit	outfit	union	reps	mcse
dich20	250	3.6	4.0	7.6	250	1.38
dich20	600	4.8	2.4	7.2	250	1.38
phq9	250	4.4	3.6	6.4	250	1.38
phq9	600	6.4	4.8	8.8	250	1.38

Marginal calibration, before the step-down. This separates a miscalibrated statistic from a miscalibrated correction: if the marginal rate is nominal and the family-wise rate is not, the studentised-max step-down is what fails for outfit, not outfit itself.

```
R |>
  filter(target == "none") |>
  group_by(arm, n) |>
  summarise(infit = round(100 * mean(p_infit < ALPHA), 1),
            outfit = round(100 * mean(p_outfit < ALPHA), 1),
            .groups = "drop")
```

arm	n	infit	outfit
dich20	250	4.7	4.2
dich20	600	4.8	4.3
phq9	250	4.4	3.8

arm	n	infit	outfit
phq9	600	5.3	4.9

Detection of the misfitting item

```
R |>
  filter(is_misfit) |>
  group_by(arm, target, loc, n) |>
  summarise(infit = round(100 * mean(f_in), 1),
            outfit = round(100 * mean(f_out), 1),
            union = round(100 * mean(f_any), 1),
            reps = n(), .groups = "drop") |>
  arrange(arm, target, n)
```

arm	target	loc	n	infit	outfit	union	reps
dich20	off	1.82	250	59.2	18.4	61.6	250
dich20	off	1.82	600	94.4	58.4	95.2	250
dich20	on	0.04	250	83.6	44.0	84.4	250
dich20	on	0.04	600	100.0	94.4	100.0	250
phq9	off	1.40	250	52.4	24.8	58.0	250
phq9	off	1.40	600	94.0	66.8	96.0	250
phq9	on	−0.15	250	69.2	55.2	71.6	250
phq9	on	−0.15	600	98.8	94.8	98.8	250

```
R |>
  filter(is_misfit) |>
  group_by(arm, target, n) |>
  summarise(infit = 100 * mean(f_in), outfit = 100 * mean(f_out),
            .groups = "drop") |>
  pivot_longer(c(infit, outfit), names_to = "statistic", values_to = "det") |>
  ggplot(aes(target, det, colour = statistic, group = statistic)) +
  geom_line() + geom_point(size = 2) +
  facet_grid(arm ~ n, labeller = label_both) +
  scale_x_discrete(limits = c("on", "off"),
                   labels = c("on target", "off target")) +
  ylim(0, 100) +
  labs(x = "Targeting of the misfitting item", y = "Detection (%)",
       colour = "Statistic") +
  theme_minimal(base_size = 10)
```

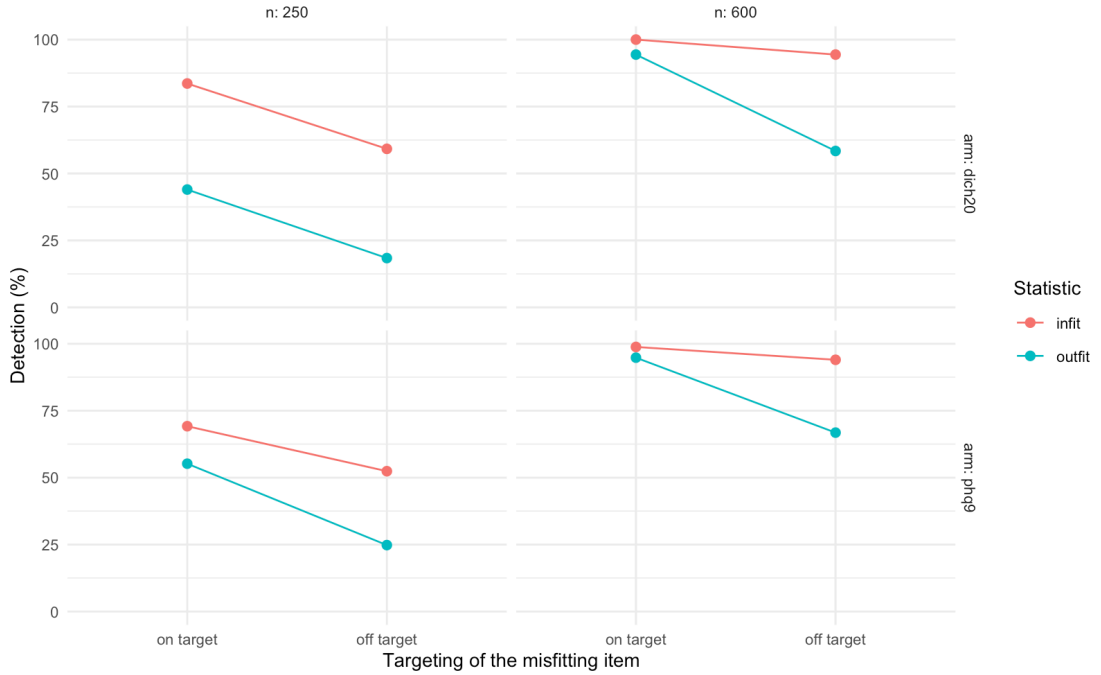

Figure 1

False positives on fitting items

Per-item rate among the items that do fit, in the conditions where one item misfits. This is the cascade check.

```
R |>
  filter(target != "none", !is_misfit) |>
  group_by(arm, target, n) |>
  summarise(infit = round(100 * mean(f_in), 2),
            outfit = round(100 * mean(f_out), 2),
            union = round(100 * mean(f_any), 2), .groups = "drop") |>
  arrange(arm, target, n)
```

arm	target	n	infit	outfit	union
dich20	off	250	0.13	0.13	0.25
dich20	off	600	0.32	0.08	0.40
dich20	on	250	0.15	0.15	0.29
dich20	on	600	0.19	0.06	0.25
phq9	off	250	0.70	0.25	0.80
phq9	off	600	1.30	0.75	1.65
phq9	on	250	0.50	0.10	0.55
phq9	on	600	1.30	0.75	1.45

Direction of the false flags, which in the earlier studies was almost entirely overfit:

```
R |>
  filter(target != "none", !is_misfit, f_in == 1 | f_out == 1) |>
```

```
  group_by(arm, target) |>
  summarise(n_flags_infit = sum(f_in),
            overfit_infit = round(100 * mean(infit_msq[f_in == 1] < 1), 1),
            n_flags_outfit = sum(f_out),
            overfit_outfit = round(100 * mean(outfit_msq[f_out == 1] < 1), 1),
            .groups = "drop")
```

arm	target	n_flags_infit	overfit_infit	n_flags_outfit	overfit_outfit
dich20	off	21	57.1	10	0.0
dich20	on	16	75.0	10	0.0
phq9	off	40	87.5	20	70.0
phq9	on	36	88.9	17	70.6

Paired discordance

The decisive table. Both statistics are computed on the same replication from the same bootstrap, so the marginal detection rates above can be identical while the two rules disagree on which replications they succeed in. `only_outfit` counts replications where outfit flags the misfitting item and infit does not, which is the whole practical case for reporting outfit alongside infit.

`mcnemar_p` is the exact McNemar test on the discordant pairs, which is `NA` when a cell has none. A cell can have no discordant pairs because both rules detect everything or because neither detects anything, so read it next to `both` and `neither` rather than alone.

```
# Exact McNemar on the discordant pairs. binom.test() errors on a + b = 0,
# which happens in any cell where the two rules agree in every replication.
exact_mcnemar <- function(a, b) {
  mapply(function(x, y) {
    if (x + y == 0L) NA_real_ else stats::binom.test(x, x + y, 0.5)$p.value
  }, a, b)
}
```

```
R |>
  filter(is_misfit) |>
  group_by(arm, target, n) |>
  summarise(both = sum(f_in == 1 & f_out == 1),
            only_infit = sum(f_in == 1 & f_out == 0),
            only_outfit = sum(f_in == 0 & f_out == 1),
            neither = sum(f_in == 0 & f_out == 0),
            reps = n(), .groups = "drop") |>
  mutate(pct_only_outfit = round(100 * only_outfit / reps, 1),
         mcnemar_p = round(exact_mcnemar(only_outfit, only_infit), 4)) |>
  arrange(arm, target, n)
```

arm	target	n	both	only_infit	only_outfit	neither	reps	pct_only_outfit	mcnemar_p
dich20	off	250	40	108	6	96	250	2.4	0e+00
dich20	off	600	144	92	2	12	250	0.8	0e+00
dich20	on	250	108	101	2	39	250	0.8	0e+00
dich20	on	600	236	14	0	0	250	0.0	1e-04
phq9	off	250	48	83	14	105	250	5.6	0e+00

arm	target	n	both	only_infit	only_outfit	neither	reps	pct_only_outfit	mcnemar_p
phq9	off	600	162	73	5	10	250	2.0	0e+00
phq9	on	250	132	41	6	71	250	2.4	0e+00
phq9	on	600	237	10	0	3	250	0.0	2e-03

Same counts under the null, where every discordant flag is a false positive on some item:

```
R |>
  filter(target == "none") |>
  group_by(arm, n, rep) |>
  summarise(i = as.integer(sum(f_in) > 0), o = as.integer(sum(f_out) > 0),
            .groups = "drop") |>
  group_by(arm, n) |>
  summarise(both = sum(i == 1 & o == 1),
            only_infit = sum(i == 1 & o == 0),
            only_outfit = sum(i == 0 & o == 1),
            neither = sum(i == 0 & o == 0), .groups = "drop")
```

arm	n	both	only_infit	only_outfit	neither
dich20	250	0	9	10	231
dich20	600	0	12	6	232
phq9	250	4	7	5	234
phq9	600	6	10	6	228

Session

```
data.frame(
  B = B, alpha = ALPHA, reps = REPS,
  cells = nrow(s$grid), elapsed_min = round(s$elapsed_min, 1),
  easyRasch2 = as.character(utils::packageVersion("easyRasch2")),
  R = paste(R.version$major, R.version$minor, sep = ".")
)
```

B	alpha	reps	cells	elapsed_min	easyRasch2	R
1000	0.05	250	3000	139.4	1.1.0.9000	4.6.1